

Self-Supervised Discovering of Interpretable Features for Reinforcement Learning

Wenjie Shi^{ID}, Gao Huang^{ID}, *Member, IEEE*, Shiji Song^{ID}, *Senior Member, IEEE*,
Zhuoyuan Wang, Tingyu Lin, and Cheng Wu^{ID}

Abstract—Deep reinforcement learning (RL) has recently led to many breakthroughs on a range of complex control tasks. However, the agent's decision-making process is generally not transparent. The lack of interpretability hinders the applicability of RL in safety-critical scenarios. While several methods have attempted to interpret vision-based RL, most come without detailed explanation for the agent's behavior. In this paper, we propose a self-supervised interpretable framework, which can discover interpretable features to enable easy understanding of RL agents even for non-experts. Specifically, a self-supervised interpretable network (SSINet) is employed to produce fine-grained attention masks for highlighting task-relevant information, which constitutes most evidence for the agent's decisions. We verify and evaluate our method on several Atari 2600 games as well as Duckietown, which is a challenging self-driving car simulator environment. The results show that our method renders empirical evidences about how the agent makes decisions and why the agent performs well or badly, especially when transferred to novel scenes. Overall, our method provides valuable insight into the internal decision-making process of vision-based RL. In addition, our method does not use any external labelled data, and thus demonstrates the possibility to learn high-quality mask through a self-supervised manner, which may shed light on new paradigms for label-free vision learning such as self-supervised segmentation and detection.

Index Terms—Deep reinforcement learning, interpretability, attention map, decision-making process

1 INTRODUCTION

OVER the last few years, deep reinforcement learning (RL) algorithms have achieved great success in a number of challenging domains, from video games [1], [2], [3] to robot navigation [4], [5], [6]. In spite of their impressive performance across a wide variety of tasks, they are often criticized for being black boxes and lack of interpretability, which has increasingly been a pressing concern in deep RL. In addition, while deep RL substantially benefits from powerful function approximation capability of deep neural networks (DNNs), poor interpretability of which further exacerbates such concerns. Hence, developing the ability to understand the agent's underlying decision-making process is crucial before using deep RL to solve real-world problems where reliability and robustness are critical.

In machine learning, there has been a lot of interest in explaining decisions of black-box systems [7], [8], [9], [10]. Some popular methods have provided visual explanations for DNNs, such as LIME [11], LRP [12], DeepLIFT [13], Grad-CAM [14], Kernel-SHAP [15] and network dissection

[16], [17]. However, these methods generally depend on class information and cannot be directly adapted to continuous RL tasks. In the context of vision-based RL, a feasible explanation approach is to learn t-Distributed Stochastic Neighbor Embedding (t-SNE) maps [1], [18], [19], but which are difficult for non-experts to understand. Moreover, there are a number of works applying gradient-based [18], [20] and perturbation-based [21] approaches to visualizing important features for RL agent's decisions, but the generated saliency maps are usually coarse and only provide limited quantitative evaluation. Another promising approach incorporates attention mechanisms into actor network to explain RL agent's decisions [22], [23], [24], [25]. However, these methods are not applicable to pretrained agent models whose internal structure cannot be changed anymore, additionally, some of these methods depend on human demonstration dataset [24], [25].

This paper aims to render reliable interpretation for vision-based RL where the agent's states are color images. To overcome the limitations of the above methods, we propose a self-supervised interpretable framework, which can discover interpretable features for easily understanding what information is task-relevant and where to look in the state. Answering these questions can provide valuable explanations about how decisions are made by the agent and why the agent performs well or badly. The main idea underlying our framework is novel and simple. Specifically, for a pretrained policy that needs to be explained, our framework learns to predict an attention mask to highlight the information (or features) that may be task-relevant in the state. If the generated actions are consistent when the policy takes as input the state and the attention-overlaid state respectively, the features highlighted

- Wenjie Shi, Gao Huang, Shiji Song, Zhuoyuan Wang, and Cheng Wu are with the Department of Automation, Tsinghua University 12442, Beijing 100084, China. E-mail: {shiwj16, zhuoyuan16}@mails.tsinghua.edu.cn, {gaohuang, shijis, wuc}@tsinghua.edu.cn.
- Tingyu Lin is with the State Key Laboratory of Intelligent Manufacturing System Technology, Beijing Institute of Electronic System Engineering, Beijing 100039, China. E-mail: lintingyu2003@sina.com.

Manuscript received 16 Mar. 2020; revised 29 Oct. 2020; accepted 9 Nov. 2020.
Date of publication 13 Nov. 2020; date of current version 1 Apr. 2022.
(Corresponding author: Gao Huang.)
Recommended for acceptance by L. Li.
Digital Object Identifier no. 10.1109/TPAMI.2020.3037898

by our framework are considered to be task-relevant and constitute most evidence for the agent's decisions.

In this paper, the kernel module of our framework, i.e., a self-supervised interpretable network (SSINet), is first presented for vision-based RL agents based on two properties, namely *maximum behavior resemblance* and *minimum region retaining*. These two properties force the SSINet to provide believable and easy-to-understand explanations for humans. After the validity is empirically proved, the SSINet is applied to further explain agents from two facets including decision-making and performance. While the former focuses on explaining how the agent makes decisions, the latter emphasizes the explanations about why the agent performs well or badly. More concretely, the agent's decisions are explained by understanding basic attention patterns, identifying the relative importance of features and analyzing failure cases. Moreover, to explain the agent's performance, such as the robustness when transferred to novel scenes, two mask metrics are introduced to evaluate the generated masks, then how the agent's attention influences performance is explained quantitatively.

We conduct comprehensive experiments on Atari 2600 video games [26] as well as Duckietown [27], which is a challenging self-driving car simulator environment. Empirical results verify the effectiveness of our method, and demonstrate that the SSINet produces high-resolution and sharp attention masks to highlight task-relevant information that constitutes most evidence for the agent's behavior. In other words, our method discovers interpretable features for easily understanding how RL agents make decisions and why RL agents perform well or badly. Overall, our method takes a significant step towards interpretable RL.

It is worth noting that our whole training procedure can be seen as self-supervised, because the data for training SSINet is collected by using the pretrained RL agent. Generally, self-supervised learning is challenging due to the lack of labelled data. It is not well understood why humans excel at self-supervised learning. For example, a child has never been supervised in pixel level, but can still perform highly precise segmentation tasks. Our method reveals a self-supervised manner to learn high-quality mask by directly interacting with the environment, which may shed light on new paradigms for label-free vision learning such as self-supervised segmentation and detection.

The remainder of this paper is organised as follows. In the following two sections, we review related works and introduce the related background of RL. In Section 4, we mainly present a self-supervised interpretable framework for vision-based RL. In Section 5, empirical results are provided to verify the effectiveness of our method. In Sections 6 and 7, our method is applied to explain how the agent makes decisions and why the agent performs well or badly, respectively. In the last section, we draw the conclusion and outline the future work.

2 RELATED WORK

2.1 Explaining Traditional RL Agents

A number of prior works [28], [29], [30] have focused on explaining traditional RL agents. For example, based on the assumption that an exact Markov Decision Process (MDP)

model is readily accessible, natural language and logic-based explanations are given for RL agents in [28] and [29] respectively. More recently, execution traces [30] of an agent are analyzed to extract explanations. However, these methods rely heavily on interpretable, high-level or hand-crafted state features, which is impractical for vision-based applications.

Other explanation methods include decision tree [31], [32], [33] and structural causal MDP [34], [35]. While decision tree can be represented graphically and thus aid in human understanding, a reasonably-sized tree with explainable attributes is difficult to construct, especially in the vision-based domain. Structural causal MDP methods are designed for specific MDP models and thus provide limited explanations.

2.2 Explaining Vision-Based RL Agents

Explaining the decision-making process of RL agents has been a particular area of interest for recent works. Here we review prior works that aim to explain how inputs influence sequential decisions in vision-based RL. Broadly speaking, existing methods can be partitioned into four categories: embedding-based methods, gradient-based methods, perturbation-based methods and attention-based methods. In addition to those works that focus on the explanation of vision-based RL, some popular and relevant works for visual explanations of DNNs will also be reviewed.

Embedding-Based Methods. The main idea underlying embedding-based methods for interpreting vision-based RL is to visualize high dimensional data with t-SNE [36], which is a commonly used non-linear dimensionality reduction method. The simplest approach is to directly map the representation of perceptually similar states to nearby points [1], [18], [19]. Each state is represented as a point in the t-SNE map, and the color of the points is set manually using global features or specific hand crafted features. In addition, there is some work attempting to learn an embedded map where the distance between any two states is related to the transition probabilities between them [37]. However, an issue with these methods is that they emphasize t-SNE clusters or state transition statistics which are uninformative to those without a machine learning background.

Gradient-Based Methods. Methods in this category aim to understand what features of an input are most salient to its output by performing only one or a few backward passes through the network. The prototypical work is Jacobian saliency maps [38] where attributions are computed as the Jacobian with respect to the output of interest. Furthermore, there are several works generating Jacobian saliency maps and presenting it above the input state to understand which pixels in the state affect the value or action prediction the most [18], [20]. Moreover, several other works modify gradients to obtain saliency maps for explanations of DNNs, such as Excitation Backpropagation [39], Grad-CAM [14], LRP [12], DeepLIFT [13] and SmoothGrad [40]. Unfortunately, Jacobian saliency maps may be difficult to interpret due to the change of manifold [21], although they are efficient to compute and have clear semantics.

Perturbation-Based Methods. This category includes methods that measure the variation of a model's output when some of the input information is removed or perturbed [41], [42]. The simplest perturbation approach computes attributions by

replacing part of an input image with a gray square [43] or region [11]. An issue with this approach is that replacing pixels with a constant color introduces undesirable information. Another popular approach estimates feature importance by probing the model with randomly masked versions of the input image, such as RISE [44]. A particular example of perturbation-based methods is Shapley values [45], but the exact computation of which is NP-hard. Thus there are recent works applying perturbation approaches to approximating Shapley values for explaining DNNs, such as LIME [11], Kernel-SHAP [15] and DASP [46].

In addition to these methods, there are some optimization-based perturbation methods [42], [47], [48] aiming to optimize for the salient region using gradient descent technique. These methods are based on similar objective functions, but the specific implementations are different. The works [42], [47] try to learn a mask to control the perturbation intensity of a Gaussian blur [42] for the input image. In fact, similar approach has been recently applied to visualize Atari agents by using masked interpolations between the original state and a Gaussian blur [21]. But a Gaussian blur perturbation fails to capture the shape of feature and results in coarse saliency maps. Another work [48] employs an encoder-decoder architecture for learning a mask to directly remove undesired regions, but it generally needs to use external labelled data and provides only limited quantitative evaluation. In this work, we build on these works and further propose a self-supervised framework to interpret RL agents both qualitatively and quantitatively, especially in terms of the underlying relationship between the agent's attention and the policy's performance, such as long-term return and generalization capability.

Attention-Based Methods. Another branch of that development is the incorporation of various attention mechanisms into vision-based RL agents. Learning attention to generate saliency maps for understanding internal decision pattern is one of the most popular methods [49] in deep learning community, and there are already a considerable number of works in the direction of interpretable RL. A mainstream approach is to augment the actor network (or agent) with customized self-attention modules [22], [50], [51], [52], [53], which learn to focus its attention on semantically relevant areas for making decisions. Another notable approach implements the key-value structure of attention to learn explainable policies by sequentially querying its view of the environment [19], [23], [54]. However, these methods generally need to change the agent's internal structure and thus cannot explain agent models that have already been trained. Moreover, there are some works attempting to build human Atari-playing attention dataset and use it to learn an explainable policy via imitation learning [24], [25], but the cost can be prohibitive and it is impractical to do that for each vision-based RL task.

3 PRELIMINARIES

We consider a standard RL setup consisting of an agent interacting with an environment E in discrete timesteps. Specifically, the agent takes an action $\mathbf{a}_t$ in a state $\mathbf{s}_t$, and receives a scalar reward r_t . Meanwhile, the environment changes its state to $\mathbf{s}_{t+1}$. We model the RL task as a Markov

TABLE 1
Summary of Notation

Notation	Description
$\mathbf{s}_t$	State at time step t
$\tilde{\mathbf{s}}_t$	Attention-overlaid state at time step t
$\mathbf{a}_t$	Action at time step t
R	Return
T	Horizon
$\mathcal{S}$	State space
$\mathcal{A}$	Action space
$N_{\mathcal{A}}$	Dimension of action space
f_e	Encoder of SSINet or feature extractor
f_d	Decoder of SSINet
f_a	Action predictor of actor network
f_{exp}	Explanation model or mask
f_{exp}^{θ}	Parameterized explanation model or mask
π_e	Expert policy
π_m	Mask policy
FOR	Feature Overlapping Rate
BER	Background Elimination Rate
$S_{e,f}$	Area of extracted features
$S_{t,f}$	Area of "true" task-relevant features
$S_{t,b}$	Area of "true" background

decision process with state space $\mathcal{S}$, action space $\mathcal{A}$, initial state distribution $p(\mathbf{s}_1)$, transition dynamics $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$, and reward function $r(\mathbf{s}_t, \mathbf{a}_t)$. In all the tasks considered here the actions are real-valued $\mathbf{a}_t \in \mathbb{R}^{N_{\mathcal{A}}}$.

An agent's behaviour is defined by a policy π , which maps states to a probability distribution over the actions $\pi: \mathcal{S} \rightarrow \mathcal{P}(\mathcal{A})$. The return from a state is defined as the sum of discounted future rewards, computed over a horizon T , i.e., $R_t = \sum_{i=t}^T \gamma^{i-t} r(\mathbf{s}_i, \mathbf{a}_i)$ with a discounting factor $\gamma \in [0, 1]$. The return depends on the actions selected, and therefore on the policy π . The goal of an agent is to learn a policy π which maximizes the following expected return from the start distribution

$$J = \mathbb{E}_{r_i, \mathbf{s}_i \sim E, \mathbf{a}_i \sim \pi} [R_0]. \quad (1)$$

In this paper, we pretrain the agent with three model-free RL algorithms including proximal policy optimization (PPO) [55], soft actor-critic (SAC) [56] and twin delayed deep deterministic policy gradient (TD3) [57]. As an on-policy method, PPO uses trust region update to improve a general stochastic policy with gradient descent. Both SAC and TD3 are off-policy and based on actor-critic architecture. While SAC generally applies soft policy gradient [58] to learn a maximum entropy policy for capturing multiple modes of near-optimal behaviour, TD3 learns a deterministic policy by building on double Q-learning [59] and deep deterministic policy gradient [60].

4 METHOD

In this section, we first present the main idea underlying our method for vision-based RL. Then, a self-supervised interpretable framework is proposed for a general RL agent. Finally, a two-stage training procedure is given for this framework. Table 1 summarizes the main notations used in this work.

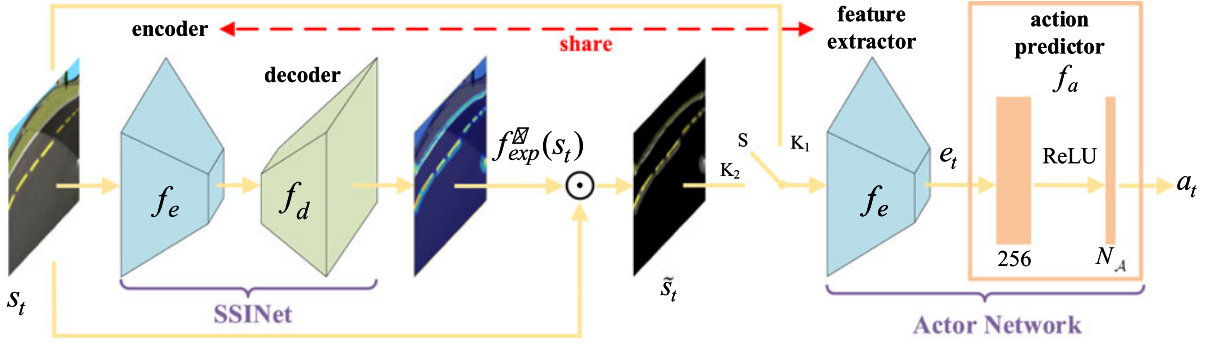


Fig. 1. Architecture diagram of our self-supervised interpretable framework, which includes two stages. In the first stage, we switch S to K_1 and jointly pretrain the feature extractor f_e and action predictor f_a with RL. In the second stage, we switch S to K_2 and train the decoder f_d while freezing f_e and f_a . The encoder is shared from the feature extractor and also denoted by f_e . N_A is the dimension of action space.

4.1 Interpreting Vision-Based RL

Consider a general setting where an expert policy is obtained by pretraining an actor (or agent) π_e , which takes as input an image s_t to predict an action. Our goal is to train a separate explanation model f_{exp} that can produce a mask $f_{\text{exp}}(s_t)$ corresponding to the importance assigned to each pixel of state s_t . In general, the mask $f_{\text{exp}}(s_t)$ can be explained as a kind of soft attention to show where the agent “looks” to make its decision. In the context of vision-based RL, the explanation model f_{exp} should satisfy two properties, namely *maximum behavior resemblance* and *minimum region retaining*.

Property 1 (Maximum behavior resemblance). For an actor (or agent) π_e and an explanation model f_{exp} , suppose $\tilde{s}_t$ is the attention-overlaid state corresponding to a specific state s_t , i.e., $\tilde{s}_t = f_{\text{exp}}(s_t) \odot s_t$, then

$$\pi_e(\tilde{s}_t) \approx \pi_e(s_t) \quad (t = 1, \dots, T), \quad (2)$$

where $\odot$ denotes the element-wise multiplication, and $\{s_1, \dots, s_T\}$ is an episode generated with π_e .

Property 2 (Minimum region retaining). For a parameterized explanation model f_{exp}^θ and a specific state s_t , the retaining region is required to be minimum after overlaying the state s_t with corresponding attention. That is

$$\min_{\theta} \left\| f_{\text{exp}}^\theta(s_t) \right\|_1, \quad (3)$$

where $\|\cdot\|_1$ denotes the L_1 -norm, and θ is the parameters of explanation model f_{exp}^θ .

Remark. Property 1 requires the agent’s behavior to be consistent with the original after the states are overlaid with the attentions generated by f_{exp} . Property 2 requires f_{exp} to attend to as little information as possible, enabling easy understanding of decision-making for humans. In addition to the above properties, we emphasize that an explanation model for vision-based RL should be able to provide valuable explanations from two facets:

Interpretability of Decision-Making. In order for an agent to be interpretable, it must not only suggest informative explanations that make sense to those without a machine learning background, but also ensure these explanations accurately represent the intrinsic reasons for the agent’s

decision-making. Concretely, it should be easy to understand how decisions are made, how an agent’s current state affects its action, what information is used and where to look. While these questions are solved, the underlying decision-making process of RL agent is partially uncovered. Note that this type of analysis does not rely on the optimal policy; if an agent takes a suboptimal or even bad action, but the reasons for which can be explained faithfully, we still consider it interpretable.

Interpretability of Performance. In the context of RL, transferability is whether the agent can generalize its policy across different scenes, and robustness is the ability of an agent to resist unknown external disturbances such as unexpected objects and new situations in novel scenes. In practice, it is meaningful and instructive to explain the performance of interest, especially when transferring the agent to novel scenes. More concretely, how the agent’s attention influences performance. What factors will affect the performance? Do the RL algorithm and actor network architecture play a major role? Answering these questions can help explain why deep RL agents perform well or badly.

4.2 Self-Supervised Interpretable Framework

In this section, we present a self-supervised interpretable framework for parameterized explanation model f_{exp}^θ . As outlined in Fig. 1, for an agent modelled by an actor network, we integrate a self-supervised interpretable network (SSINet) in front of the actor network. While the agent receives a state to predict an action at each time step, the SSINet produces an attention mask to highlight the task-relevant information for making decision without any external supervised signal. To that end, the SSINet must learn which parts of the state are considered important by the agent.

SSINet. Learning the mask is a dense prediction task, which arises in many vision problems, such as semantic segmentation [61], [62] and scene depth estimation [63]. Most of those approaches adopt an encoder-decoder structure. In order to make our masks sharp and precise, we build our SSINet by directly adapting a U-Net architecture [61] with only minor changes, exact details of which are described in Appendix A of the supplemental material, which can be found on the Computer Society Digital Library at <http://doi.ieeecomputersociety.org/10.1109/TPAMI.2020.3037898>.

As depicted in Fig. 1, our SSINet includes an encoder f_e and a

decoder f_d . Specifically, a state $\mathbf{s}_t \in \mathbb{R}^{H \times W \times C}$ at time step t (here a frame of height H , width W and channel C) is encoded through f_e to obtain low-resolution feature map, which is then taken as input of f_d and upsampled to produce an attention mask $f_{\text{exp}}^\theta(\mathbf{s}_t) \in [0, 1]^{H \times W \times 1}$

$$f_{\text{exp}}^\theta(\mathbf{s}_t) = \sigma(f_d(f_e(\mathbf{s}_t))), \quad (4)$$

where $\sigma(\cdot)$ is the sigmoid nonlinearity. Afterwards, the attention mask $f_{\text{exp}}^\theta(\mathbf{s}_t)$ is broadcast along the channel dimension of the state $\mathbf{s}_t$, and element-wise multiplied with it to form a masked (or attention-overlaid) state $\tilde{\mathbf{s}}_t$

$$\tilde{\mathbf{s}}_t = f_{\text{exp}}^\theta(\mathbf{s}_t) \odot \mathbf{s}_t, \quad (5)$$

where $\odot$ denotes the element-wise multiplication.

Actor Network. Generally, we use an actor network to model the policy of an RL agent. As shown in Fig. 1, the actor network includes a *feature extractor* f_e and an *action predictor* f_a . The *feature extractor* f_e takes as input a masked state $\tilde{\mathbf{s}}_t$ (or a state $\mathbf{s}_t$), and outputs low-resolution feature map $\mathbf{e}_t$. The *action predictor* f_a is a simple two-layer perception, which uses flatten feature map to predict an action $\mathbf{a}_t \in \mathbb{R}^{N_A}$. For clarify, we denote by *expert policy* π_e the actor network taking as input $\mathbf{s}_t$, and denote by *mask policy* π_m the actor network taking as input $\tilde{\mathbf{s}}_t$

$$\pi_e(\mathbf{s}_t) = \phi(f_a(f_e(\mathbf{s}_t))), \quad (6)$$

$$\pi_m(\mathbf{s}_t) = \phi(f_a(f_e(\tilde{\mathbf{s}}_t))), \quad (7)$$

where $\phi(\cdot)$ is the tanh nonlinearity for continuous RL tasks or the softmax nonlinearity for discrete RL tasks.

There are several advantages of our self-supervised interpretable framework. First, our interpretable framework is applicable to any RL model taking as input visual images. Second, the SSINet learns to predict task-relevant information, which can provide intuitive and valuable explanations for the agent's decisions. Finally, we emphasize that the SSINet is a flexible explanatory module which can be adapted to other vision-based decision-making systems.

Algorithm 1. Two-Stage Training Procedure for SSINet

- 1 Initialize f_e, f_d and f_a ;
 - 2 Switch S to K_1 , obtain an expert policy π_e by jointly pretraining f_e and f_a with RL;
 - 3 Generate state-action pairs $\{(\mathbf{s}_i, \mathbf{a}_i^l)\}_{i=1}^M$ using π_e ;
 - 4 Switch S to K_2 , freeze f_e and f_a ;
 - 5 **for each iteration do**
 - 6 Calculate the mask loss L_{mask} (8) with collected state-action pairs;
 - 7 Update f_d using L_{mask} ;
 - 8 **end**
-

4.3 Training Procedure

Our training procedure includes two stages. The first stage aims to obtain an expert policy π_e and then use it to generate state-action pairs, which is used for training SSINet in the second stage. The objective of second stage is to learn interpretable attention masks f_{exp}^θ for explaining the agent's

behaviour. Instead of learning f_{exp}^θ from scratch, we reuse the *feature extractor* f_e of actor network as the encoder of SSINet. This offers at least two advantages: (i) it speeds up the training since internal features from the RL agent can be reused; (ii) it greatly reduces the risk of overfitting and ensures that the generated mask is compatible with the RL agent to be explained. Overall, the whole training procedure is self-supervised, since there is no external labelled data.

In the first stage, we switch S to K_1 (as shown in Fig. 1) and pretrain the *feature extractor* f_e and *action predictor* f_a with PPO [55] algorithm. After training, the resulting expert policy π_e is used to collect data by generating M state-action pairs $\{(\mathbf{s}_i, \mathbf{a}_i^l)\}_{i=1}^M$ with the action $\mathbf{a}_i^l = \pi_e(\mathbf{s}_i)$.

In the second stage, we switch S to K_2 and train the SSINet. The weights of the encoder is fixed during the second stage. Based on Property 1, our goal is to learn attention masks such that there is minimum variation between the expert policy (6) and mask policy (7). Moreover, Property 2 requires the learned mask to attend to as little information as possible. These considerations lead to the mask loss function as follows:

$$L_{\text{mask}} = \sum_{i=1}^N \frac{1}{2} \|\pi_m(\mathbf{s}_i) - \pi_e(\mathbf{s}_i)\|_2^2 + \alpha \|f_{\text{exp}}^\theta(\mathbf{s}_i)\|_1, \quad (8)$$

where $\|\cdot\|_1$ denotes the L_1 -norm, α is a positive scalar controlling the sparseness of the mask, and N is the batch size. The first term ensures that the agent's behaviour does not change much after overlaying the state with corresponding attention mask, while the second term is a sparse regularization that pushes for better visual explanations for humans.

One point worth noting is that only the *decoder* f_d is trained in the second stage, since the *encoder* f_e of SSINet is shared from the *feature extractor* of actor network, which is pretrained in the first stage. The pseudo-code of our training procedure is summarized in Algorithm 1.

5 VALIDITY OF OUR METHOD

Before we apply the proposed method to provide explanations for vision-based RL, we first verify the effectiveness of our method through performance evaluation and comparative evaluation in this section. Then, our method is applied to render empirical evidences about how the agent makes decisions in Section 6 and why the agent performs well or badly in Section 7. The code and implementation details are available at: <https://github.com/shiwj16/SSINet>.

5.1 Setup

We verify and evaluate the performance of our method on several Atari 2600 games and Duckietown environment (see below for details). All experimental details are given in Appendix C of the supplemental material, available online. Note that during data collection, the expert policy π_e is used to generate $M = 50000$ state-action pairs for training the SSINet.

Duckietown. Duckietown [27] is a self-driving car simulator environment for OpenAI Gym [64]. It places the agent, a Duckiebot, inside of an instance of a Duckietown: a loop of roads with turns, intersections, obstacles, and so on. Specifically, the states are selected as $120 \times 160 \times 3$ color images

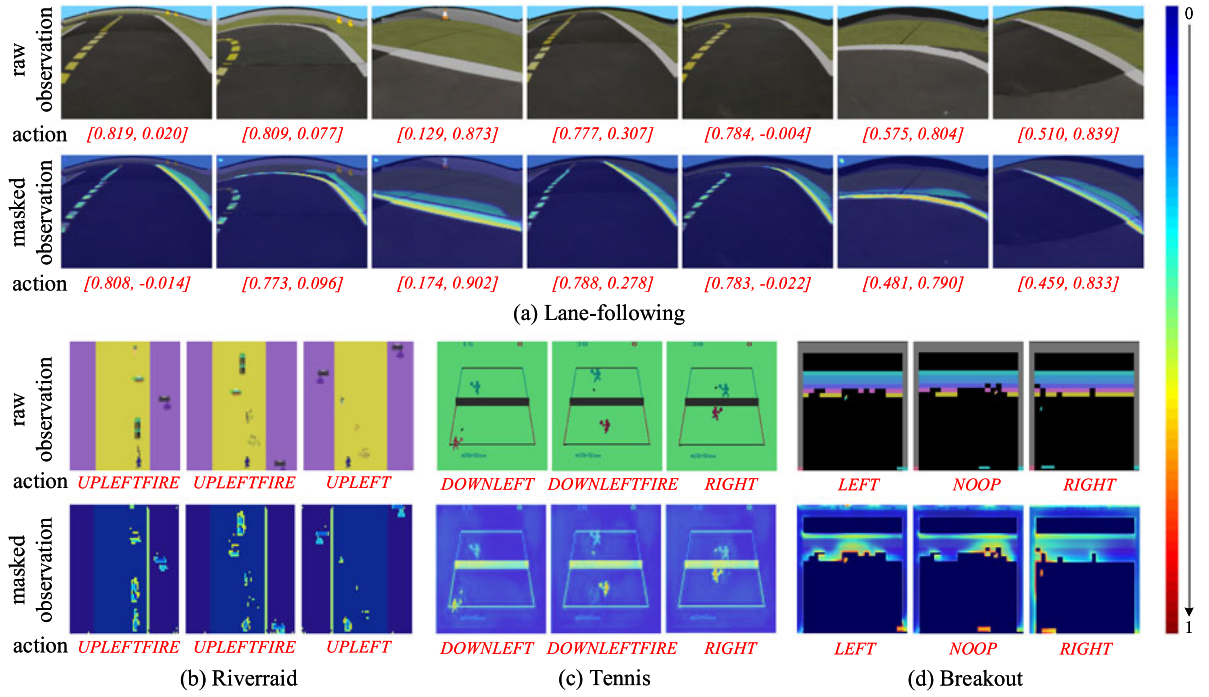


Fig. 2. Basic attention patterns (time goes from left to right). For each task, the top row shows a sequence of state-action pairs generated by the expert policy while the bottom row shows a sequence of masked state-action pairs generated by the mask policy. Bright areas in heatmaps are the regions used to make decisions by the mask policy. Best viewed on a computer monitor.

which are resized from single camera images with $480 \times 640 \times 3$, and the actions contain two continuous normalized numbers corresponding to forward velocity and steering angle respectively. The goal of an agent is to drive forward along the right lane, hence the agent will be rewarded for being as close as possible to the center line of the lane, and also for facing the same direction as the lane's tangent. In our experiments, we evaluate our method on Lane-following task, and the *empty* map is chosen as the training scene. In addition to *empty* and *zigzag* maps provided by the official environment, another eight customized maps (including *empty-city*, *zigzag-city*, *corner*, *corner-city*, *U-turn*, *U-turn-city*, *S-turn* and *S-turn-city*) are designed only for evaluation. These ten maps are mainly different from each other in background and driving route, detailed descriptions are given in Appendix B of the supplemental material, available online.

Atari 2600. In addition to Duckietown, we also perform experiments in the Arcade Learning Environment (ALE) [26], which is a commonly used benchmark environment for discrete RL tasks. In our experiments, we select several challenging games, such as Tennis, Riverraid and Breakout. In each game, the agent receives $84 \times 84 \times 4$ stacked gray-scale images as inputs, as described in [1]. It is worth noting that the Riverraid and Breakout tasks have no strong pattern of large “super pixels”. While the agent uses the boundary for attention as opposed to wide lane on Riverraid task, the task-relevant features on Breakout task have non-trivial patterns, which make super-pixel based methods (such as flood-fill) impossible. Tennis is a singles tennis game where direct implementation of our method may fail, since the task-relevant feature happens to be black. To tackle this problem, we change the color of black features by adding a small value to each pixel of the state when applying our method to the Tennis task.

5.2 Evaluations

To demonstrate the effectiveness of our proposed method, we verify the RL agent's behaviour consistency between mask policy and expert policy from two aspects, i.e., *average return* and *behavior matching*. *Average return* represents the long-term rewards of two policies, while *behavior matching* characterizes the behavioural similarity of two policies. We note that similar metrics are also used for attention-guided learning in recent work [25].

Fig. 2 shows the results of our SSINet on four tasks in terms of *behavior matching*. It can be observed that the mask policy makes decisions using partial information (or the bright areas in the heatmaps) while the expert policy uses all information in the state, but as expected, the mask policy predicts almost the same actions as the expert policy. This observation verifies that the attention masks produced by our SSINet can accurately highlight the task-relevant information, which constitutes most evidence for the expert policy's behaviour.

To further verify the effectiveness of our method, we compare the performance of expert policy and mask policy in terms of *average return*, as shown in Figs. 3 and 4. It can be seen that the mask policy consistently achieves greater long-term rewards than the expert policy on all maps except the *S-turn-city* map. As stated in Section 4.2, the expert policy and mask policy take as input original state and attention-overlaid state, respectively. Therefore, we can conclude that the attention masks produced by SSINet can retain task-relevant information and filter out task-irrelevant information. Fig. 5 visualizes the performance on *empty*, *empty-city*, *zigzag* and *zigzag-city* maps, and more visualization results are given in Appendix D of the supplemental material, available online.

Comparative Evaluation. We compare our method against several popular RL explanation methods including Gradient-

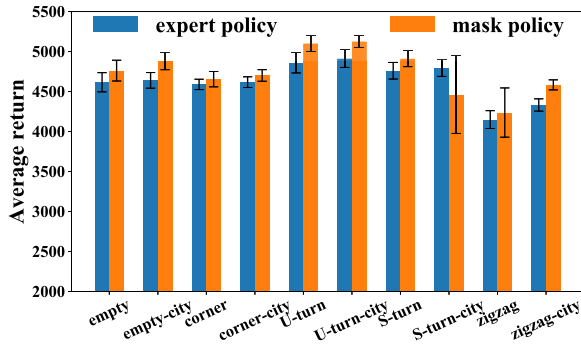


Fig. 3. Performance comparisons between expert policy and mask policy on Lane-following task. Both two policies are evaluated on ten maps. All returns are averaged across 15 evaluation episodes, and the errorbars represent the standard variations.

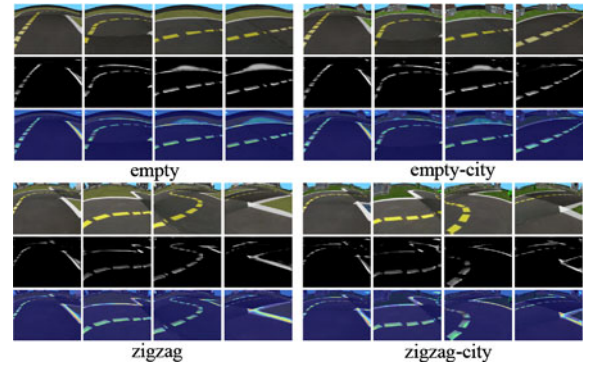


Fig. 5. Visualization of the performance on four maps. For each map, three rows represent a sequence of states, attention masks and masked states respectively (time goes from left to right).

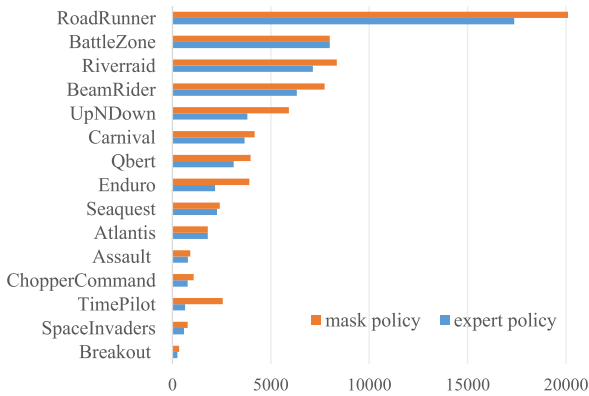


Fig. 4. Performance comparisons between expert policy and mask policy on Atari games.

based saliency method (Gradient-Saliency) [18] and Gaussian perturbation-based saliency method (Perturbation-Saliency) [21]. These methods focus on the interpretability of vision-based RL and have been briefly reviewed in Section 2. Fig. 6

compares our method with Jacobian-based and Perturbation-based methods in terms of the quality of saliency map. The results show that our method produces higher-resolution and sharper saliency maps than the others. Moreover, the saliency maps generated by our method can reflect the relative importance of features by the depth of color. In contrast, the other methods can only locate the position of features. Note that the blue in the last two rows of Fig. 6 does not correspond to saliency value.

To quantitatively evaluate the quality of masks generated by different methods, we further compare the average return of several policies, which can access to only the pixels of particular masks obtained by different methods during the training. Table 2 summarizes the average return of three different methods, and all results are averaged across five random training runs. It can be seen that the policy still achieves good performance when accessing to only the pixels of the mask generated by our method. In contrast, we can observe obvious performance degradation when using the other methods to generate the masks.

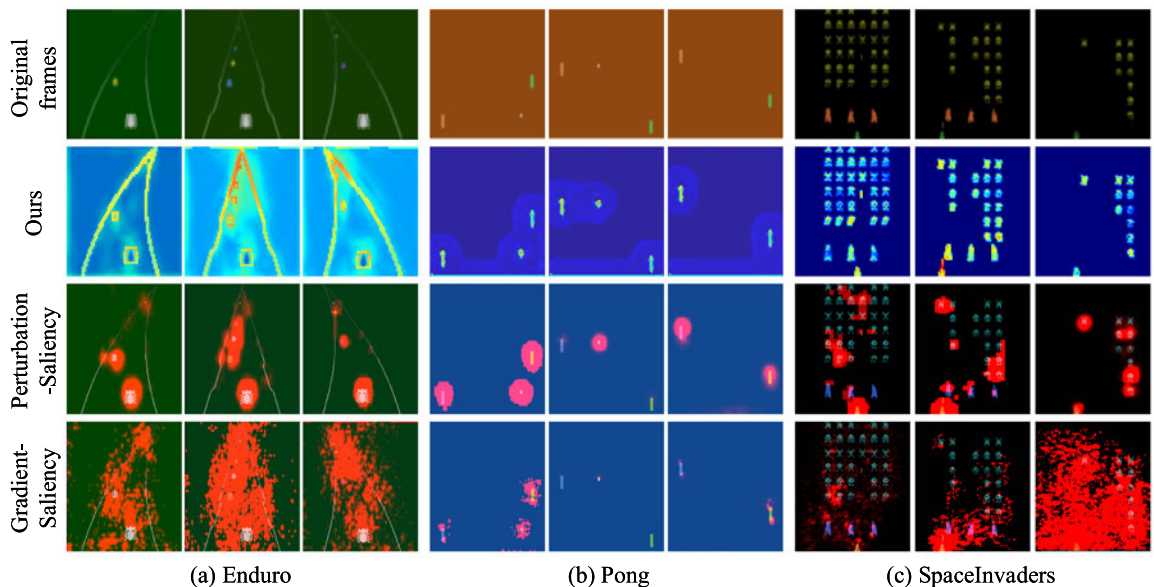


Fig. 6. Comparison between our method and other methods, including Gradient-based saliency method (Gradient-Saliency) [18] and Gaussian perturbation-based saliency method (Perturbation-Saliency) [21]. For each method, a short sequence of saliency-overlaid states is shown.

TABLE 2
The Comparison of Average Returns When the Policy is Learned With Access to Only the Pixels of Particular Masks Obtained by Different Methods During the Training

Tasks	Ours	Perturbation method [21]	Gradient method [18]
Enduro	2755.42	1741.10	819.92
Seaquest	2356.67	1830.00	836.00
SpaceInvaders	740.45	562.92	297.73

6 INTERPRETING THE DECISION OF AGENTS

In this section, our method is applied to explain *how RL agent makes decisions*. First, basic attention patterns for making decisions are recognized and understood. Second, the relative importance of different task-relevant features is identified for easy understanding of the agent's decision-making process. Third, some failure cases are analyzed from the viewpoint of *attention shift*.

6.1 Basic Attention Patterns for Making Decisions

Here we explain how RL agent makes decisions by understanding the agent's basic attention patterns. As visualized in Fig. 2, the most dominant pattern is that the agent focuses on only small regions which are strongly task-relevant, while other regions are very "blurry" and can be ignored. In other words, the state is not a primitive, the agent learns what information is important for making decisions and where to look at each time step. For example, the task-relevant features are white edge line and yellow dashed line on Lane-following task, players, tennis net and ball on Tennis task. In fact, this conclusion is consistent with human gaze-action pattern [65], one characteristic of which is that humans tend to focus attention selectively on parts of the visual space to acquire task-relevant information when and where it is needed.

6.2 Relative Importance of Task-Relevant Features

In addition to making it clear what information is used and where to look, understanding the relative importance of different task-relevant features is also crucial for easily explaining the agent's decision-making process. Although the value of attention mask (or the depth of color in heatmap) has intuitively indicated the relative importance of different features in the state, it is not strictly verifiable.

In this section, we seek to identify the relative importance of task-relevant features in a more interpretable way. We observe that greater regularization scale α in mask loss (8) actually means severer penalty to the agent for attending to task-irrelevant regions. Based on this observation, we propose to assess the relative importance of task-relevant features by comparing multiple attention masks trained with different values of α . To that end, we perform a fine search to visualize the evolving process of attention masks. Fig. 7 shows the evolution of attention masks in the form of heatmap as the regularization scale α varies.

As can be seen in Fig. 7, with the increasing of regularization scale, the "attended" regions are gradually narrowed down to the most important information as expected. Concretely, the inner side of both yellow dashed line and white

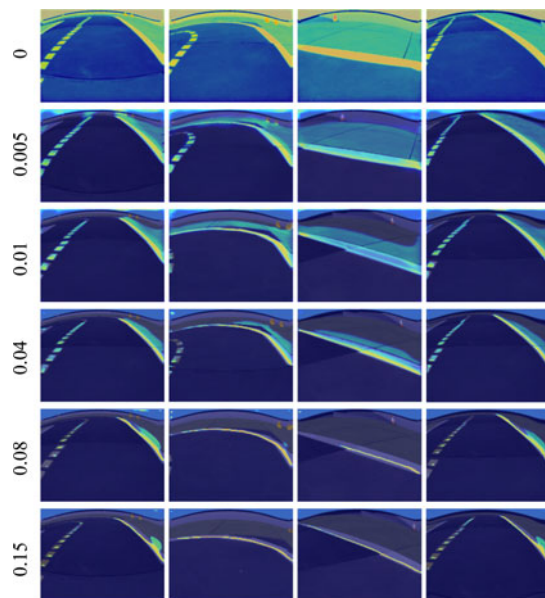


Fig. 7. Evolution of the agent's attention as the regularization scale α varies. Bright areas are the "attended" regions for making decisions.

edge line is considered to be more important than the outer side for making decisions, and nearby lines are considered to be more important than distant lines. In fact, this conclusion is consistent to human gaze system where limited visual sensor resources will be assigned to the most important information.

6.3 Analysis of Failure Case

In practice, it is critical to ensure that a trained RL agent can be directly transferred to novel scenes different from the scene for training. However, robustness is not always guaranteed. Take Lane-following task for example, as can be seen in Fig. 3, there is a significant performance degradation when transferring the agent trained on *empty* map to other maps, such as *S-turn-city*, *zigzag* and *zigzag-city*. This robustness problem can be explained intuitively from the point of view of *attention shift*.

In those failure cases, we notice that the agent is prone to divert its attention from task-relevant information to background when facing some novel situations, this phenomenon is called *attention shift*. Fig. 8 visualizes a common problematic situation leading to poor robustness in *S-turn-city* and *zigzag-city* maps, in which the agent needs to turn

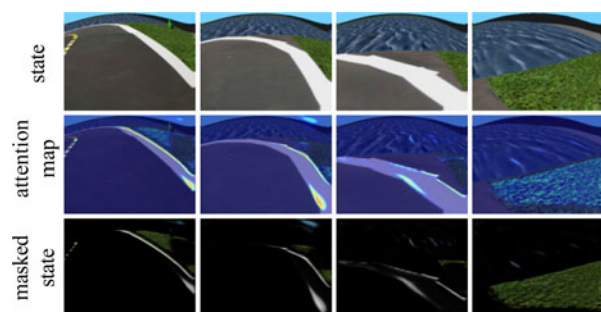


Fig. 8. A problematic situation (time goes from left to right). Three rows correspond to original states, generated attention maps and masked states, respectively.

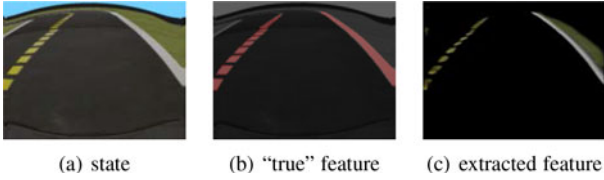


Fig. 9. An example for calculating the mask metrics. (a) is a specific state from Duckietown, red area in (b) is the “true” task-relevant features that we expect to learn, and unmasked area in (c) is the features extracted by our SSINet.

left on a corner surrounded by the grassland and lake. However, this novel situation has never been encountered on *empty* map when training, hence it may be difficult for the agent to judge what features are important for making decisions in current situation. As a result, the agent gradually loses attention to task-relevant information (white edge line) and mistakenly attends to the background (lake and grassland), then it gets into stuck due to catastrophic cumulative attention shift.

7 INTERPRETING THE PERFORMANCE OF AGENTS

In this section, our method is applied to explain *why the agent performs well or badly* quantitatively, especially when transferred to novel scenes. To that end, we start with introducing two evaluation metrics to assess the attention masks generated by SSINet. These metrics allow us to give quantitative explanation about how the agent’s attention influences its performance. Then, they are further used to explain the performance of RL agents trained with different algorithms and actor architectures. Finally, potential extension of our method to self-supervised learning is briefly discussed.

7.1 Mask Evaluation Metrics

To interpret the performance of RL agents, two evaluation metrics are introduced to assess the quality of generated attention masks. Specifically, *feature overlapping rate* and *background elimination rate* are defined as:

- *Feature Overlapping Rate* (FOR) - the overlapping ratio between the area of true mask and learned mask.
- *Background Elimination Rate* (BER) - the ratio of eliminated background area by the mask to the whole background area.

For a specific state s , mask metrics FOR(s) and BER(s) are calculated as follows:

$$\text{FOR}(s) = \frac{S_{e,f} \cap S_{t,f}}{S_{t,f}}, \quad (9)$$

$$\text{BER}(s) = \frac{S_{t,b} - S_{t,b} \cap S_{e,f}}{S_{t,b}}, \quad (10)$$

where $\cap$ and $\cup$ are union and intersection operators respectively. $S_{e,f}$, $S_{t,f}$ and $S_{t,b}$ represent the area of extracted features, “true” task-relevant features and “true” background respectively, as shown in Fig. 9. Note that the “true” background $S_{t,b}$ is the area outside the “true” features $S_{t,f}$ in Fig. 9b. In general, FOR indicates how agents can extract

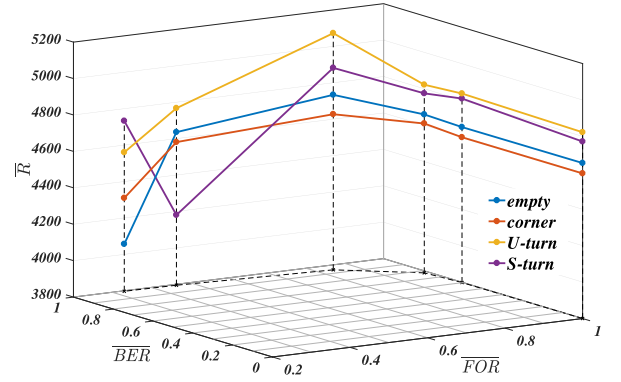


Fig. 10. Relationship between the average return $\bar{R}$ and the average mask metrics (FOR and BER). FOR and BER are the average results across 10 random states. Each black node in the FOR – BER plane corresponds to a mask policy.

useful information from the state and BER indicates how the SSINet can eliminate task-irrelevant information in the state. We point out that our FOR and BER metrics are equivalent to *true positive rate* (TPR) and *true negative rate* (TNR) from the perspective of pixel classification, and can be readily converted to TPR and TNR. Nevertheless, we prefer to use FOR and BER to evaluate the quality of the generated masks in the field of RL, since they have a more intuitive meaning and enable easy understanding of RL agents even for non-experts.

However, it is difficult to obtain the ground-truth feature, which is required to compute FOR and BER. To address this problem, we select Duckietown as the main environment for quantitative evaluation, and use prior information to manually annotate “true” features that are similar to what humans expect. Specifically, we annotate the right edge line and center dashed line as the “true” features due to two reasons. First, the goal of Duckietown tasks is to encourage the agent to drive forward along the right lane, and the agent is rewarded for being as close as possible to the center line of the right lane according to the official guidance,¹ hence it is reasonable to treat the right edge line and center dashed line as the “true” features. Second, these “true” features are consistent with human driving experience. In our experiments, we use averaged mask metrics $\overline{\text{FOR}}$ and $\overline{\text{BER}}$ on the training map to characterize the attention masks generated with our SSINet.

7.2 How the Agent’s Attention Influences Performance

In order to analyze quantitatively how vision-based RL agent’s attention influences its performance, especially when transferred to novel scenes, we compare the average returns $\bar{R}$ of multiple mask policies. These mask policies are all trained to interpret identical RL agent but produce different attention masks for the same state. In our experiments, we train SSINet under different regularization scales α for the same actor network.

Fig. 10 shows how the average mask metrics ($\overline{\text{FOR}}$ and $\overline{\text{BER}}$) influence the average return $\bar{R}$ on four maps. As can

1. <https://github.com/duckietown/gym-duckietown>

TABLE 3
 R^2 Correlation Value (Pearson) Between the Average Mask Metrics and the Average Return $\bar{R}$

Correlation value R^2 (Pearson)	Duckietown tasks			
	<i>empty</i>	<i>corner</i>	<i>U-turn</i>	<i>zigzag</i>
$R^2(\overline{BER}, \bar{R})$	0.086	0.081	0.006	0.017
$R^2(\overline{FOR}, \bar{R})$	0.525	0.550	0.367	0.271
$R^2(\overline{F1}, \bar{R})$	0.021	0.018	0.148	0.161
$R^2(\overline{FOR} \times \overline{BER}, \bar{R})$	0.655	0.699	0.760	0.504
$R^2(\overline{AUC}, \bar{R})$	0.674	0.715	0.812	0.559

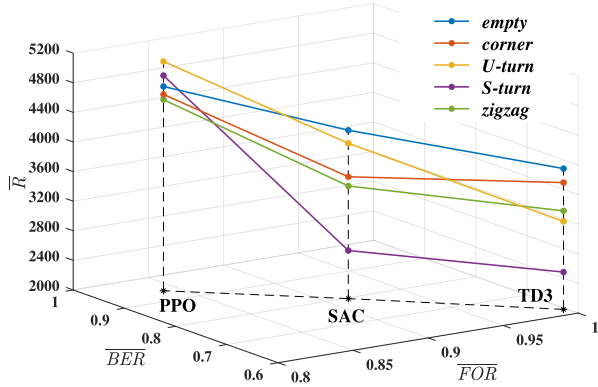


Fig. 11. The performance of PPO, SAC and TD3 agents. For each algorithm, the black node in the horizontal plane represents the mask metrics $\overline{FOR}$ and $\overline{BER}$, while the vertical axis represents the average return $\bar{R}$ of corresponding mask policy.

be seen in Fig. 10, when evaluated on the same map, the agent performs differently with regards to different $\overline{FOR} - \overline{BER}$. Only when both $\overline{FOR}$ and $\overline{BER}$ have high values, the best performance can be achieved. In other words, the agent can not perform well enough if it neglects task-relevant information or attends too much to the background, reflected either by a small $\overline{FOR}$ or a small $\overline{BER}$.

Table 3 presents the R^2 correlation value (Pearson) between the average mask metrics ($\overline{FOR}$, $\overline{BER}$, F1-score and $\overline{AUC}$) and the average return $\bar{R}$ on four different Duckietown maps. It can be seen that while there is a weak correlation between single metric ($\overline{FOR}$ or $\overline{BER}$) and return, the average return is strongly correlated with $\overline{FOR} \times \overline{BER}$ or $\overline{AUC}$. This observation reveals that $\overline{FOR}$ and $\overline{BER}$ are not independent of each other, and an agent can get good performance only when both $\overline{FOR}$ and $\overline{BER}$ are great enough. In fact, this conclusion is consistent with Fig. 10.

7.3 Explaining the Performance of Different Agents

Generally, RL agents may exhibit different performance even on simple tasks. To explain why the agent performs well or badly, especially in terms of stability and robustness when transferred to novel scenes, the above mask metrics are utilized to analyze the behaviour of multiple RL algorithms and actor architectures. Such an analysis can provide explainable basis for the selection of models and actor architectures.

Case 1: RL Algorithm. To explain the performance difference of RL algorithms, we analyze the average return $\bar{R}$ of three popular RL algorithms (PPO, SAC and TD3) with the above mask metrics. As shown in Fig. 11, PPO consistently

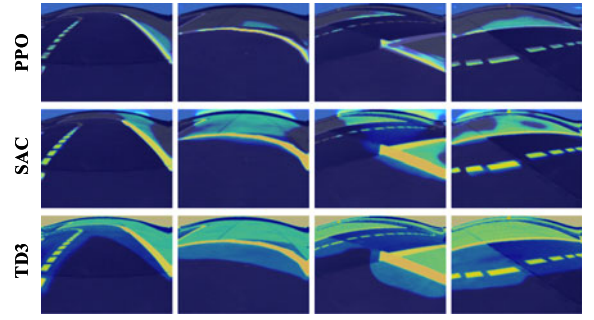


Fig. 12. Visualization of performance for PPO, SAC and TD3 algorithms. Scenes in each column are the same.

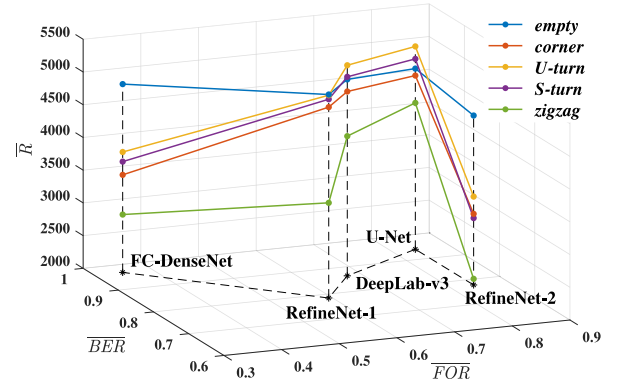


Fig. 13. The performance of five agents with different actor network architectures. For each architecture, the black node in the horizontal plane represents the mask metrics $\overline{FOR}$ and $\overline{BER}$, while the vertical axis represents the average return $\bar{R}$ of corresponding mask policy.

outperforms both SAC and TD3 on all maps for Lane-following task. The reason for this is the background information has adverse effect on the agent's performance, and our mask metrics can help quantify it. Specifically, although the $\overline{FOR}$ of SAC and TD3 are close to one indicating that almost all task-relevant information are identified, a large amount of background information is also mistakenly attended to due to small $\overline{BER}$. In contrast, PPO focuses on main task-relevant information while masking most background information. These conclusions are illustrated and further verified by Fig. 12, which visualizes the performance of PPO, SAC and TD3. Moreover, we observe that PPO shows better stability than SAC and TD3. Concretely, while PPO agent tends to drive smoothly in the center line of right lane, both SAC and TD3 agents have obvious lateral deviation and drive unsteadily.

Case 2: Actor Architecture. To understand how the actor architecture affects the agent's performance, we analyze the average return $\bar{R}$ of four popular semantic segmentation architectures (U-Net [61], RefineNet² [66], FC-DenseNet [67] and DeepLab-v3 [62]) with the above mask metrics. Detailed descriptions of these architectures can be found in Appendix A of the supplemental material, available online. As shown in Figs. 13 and 14, the performance difference among five agents results from diverse attention behaviours

2. Notice that RefineNet-1 and RefineNet-2 are exactly the same except for utilizing ResNet and MobileNet as the backbone network respectively.

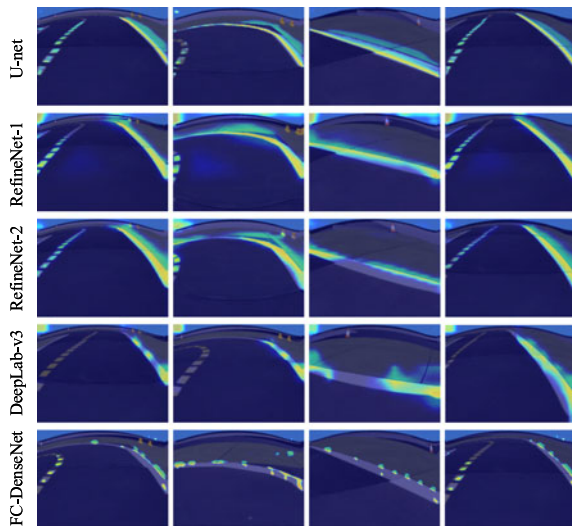


Fig. 14. Visualization of performance for Unet, RefineNet, DeepLab-v3 and FC DenseNet. Each row is a sequence of masked states (time goes from left to right). Scenes in each column are similar.

indicated by $\overline{\text{FOR}}$ and $\overline{\text{BER}}$. Losing a lot of task-relevant information (corresponding to small $\overline{\text{FOR}}$ such as FC-DenseNet) or mistakenly matching much background (corresponding to small $\overline{\text{BER}}$ such as RefineNet-1 and RefineNet-2) leads to bad performance. Moreover, we conclude that complex actor architecture does not necessarily lead to good performance. In fact, task-relevant features are relatively simple in the context of RL, hence most DNN architectures can extract them well. In practice, small DNNs are generally capable of learning representations and preferred to make the training algorithm focus on the credit assignment problem.

7.4 Potential Extension to Self-Supervised Learning

As a relatively recent learning technique in machine learning, self-supervised learning is challenging due to the lack of labelled data. In our work, we presented a self-supervised interpretable framework for vision-based RL, and a two-stage training procedure is applied to train the SSINet in a self-supervised manner. The learning signal is acquired through the direct interaction between RL agent and environment, and the whole training process is completely label-free. Empirical results in Figs. 2 and 5 demonstrate that our method is capable of learning high-quality mask through the direct interaction with the environment and without any external supervised signal. In summary, our work may shed light on new paradigms for label-free vision learning such as self-supervised segmentation and detection.

8 CONCLUSION

In this paper, we addressed the growing demand for human-interpretable vision-based RL from a fresh perspective. To that end, we proposed a general self-supervised interpretable framework, which can discover interpretable features for easily understanding the agent's decision-making process. Concretely, a self-supervised interpretable network (SSINet) was employed to produce high-resolution and sharp attention masks for highlighting task-relevant

information, which constitutes most evidence for the agent's decisions. Then, our method was applied to render empirical evidences about how the agent makes decisions and why the agent performs well or badly, especially when transferred to novel scenes. Overall, our work takes a significant step towards interpretable vision-based RL. Moreover, our method exhibits several appealing benefits. First, our interpretable framework is applicable to any RL model taking as input visual images. Second, our method does not use any external labelled data. Finally, we emphasize that our method demonstrates the possibility to learn high-quality mask through a self-supervised manner, which provides an exciting avenue for applying RL to self automatically labelling and label-free vision learning such as self-supervised segmentation and detection.

ACKNOWLEDGMENTS

This work was supported in part by the National Science and Technology Major Project of the Ministry of Science and Technology of China under Grants 2018AAA0100701 and 2018YFB1702903, the National Natural Science Foundation of China under Grants 61906106, 61936009, and 62022048, the Institute for Guo Qiang of Tsinghua University and Beijing Academy of Artificial Intelligence. The authors would like to thank the reviewers for their valuable comments.

REFERENCES

- [1] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, 2015, Art. no. 529.
- [2] D. Silver *et al.*, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, no. 7578, 2016, Art. no. 484.
- [3] W. Shi, S. Song, H. Wu, Y.-C. Hsu, C. Wu, and G. Huang, "Regularized anderson acceleration for off-policy deep reinforcement learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 10 231–10 241.
- [4] P. Mirowski *et al.*, "Learning to navigate in complex environments," in *Proc. Int. Conf. Learn. Representations*, 2017.
- [5] W. Shi, S. Song, C. Wu, and C. L. P. Chen, "Multi pseudo Q-learning-based deterministic policy gradient for tracking control of autonomous underwater vehicles," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 12, pp. 3534–3546, Dec. 2019.
- [6] W. Shi, S. Song, and C. Wu, "High-level tracking of autonomous underwater vehicles based on pseudo averaged Q-learning," in *Proc. IEEE Int. Conf. Syst., Man, Cybern.*, 2018, pp. 4138–4143.
- [7] Q. Cao, X. Liang, B. Li, and L. Lin, "Interpretable visual question answering by reasoning on dependency trees," *IEEE Trans. Pattern Anal. Mach. Intell.*, early access, Sep. 24, 2019, doi: [10.1109/TPAMI.2019.2943456](https://doi.org/10.1109/TPAMI.2019.2943456).
- [8] M. Monfort *et al.*, "Moments in time dataset: One million videos for event understanding," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 2, pp. 502–508, Feb. 2020.
- [9] H. Liu, R. Wang, S. Shan, and X. Chen, "What is Tabby? Interpretable model decisions by learning attribute-based classification criteria," *IEEE Trans. Pattern Anal. Mach. Intell.*, early access, Nov. 20, 2019, doi: [10.1109/TPAMI.2019.2954501](https://doi.org/10.1109/TPAMI.2019.2954501).
- [10] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A survey of methods for explaining black box models," *ACM Comput. Surv.*, vol. 51, no. 5, 2019, Art. no. 93.
- [11] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you? Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 1135–1144.
- [12] A. Binder, G. Montavon, S. Lapuschkin, K.-R. Müller, and W. Samek, "Layer-wise relevance propagation for neural networks with local renormalization layers," in *Proc. Int. Conf. Artif. Neural Netw.*, 2016, pp. 63–71.
- [13] A. Shrikumar, P. Greenside, A. Shcherbina, and A. Kundaje, "Learning important features through propagating activation differences," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 3145–3153.

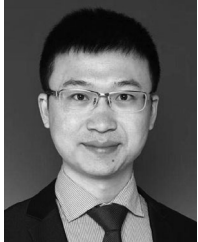
- [14] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 618–626.
- [15] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 4765–4774.
- [16] D. Bau, B. Zhou, A. Khosla, A. Oliva, and A. Torralba, "Network dissection: Quantifying interpretability of deep visual representations," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 3319–3327.
- [17] B. Zhou, D. Bau, A. Oliva, and A. Torralba, "Interpreting deep visual representations via network dissection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 9, pp. 2131–2145, Sep. 2019.
- [18] T. Zahavy, N. Ben-Zrihem, and S. Mannor, "Graying the black box: Understanding DQNs," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1899–1908.
- [19] R. M. Annasamy and K. Sycara, "Towards better interpretability in deep Q-networks," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 4561–4569.
- [20] Z. Wang, T. Schaul, M. Hessel, H. Van Hasselt, M. Lanctot, and N. De Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1995–2003.
- [21] S. Greydanus, A. Koul, J. Dodge, and A. Fern, "Visualizing and understanding atari agents," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1787–1796.
- [22] A. Manchin, E. Abbasnejad, and A. V. D. Hengel, "Reinforcement learning with attention that works: A self-supervised approach," in *Proc. Int. Conf. Neural Inf. Process.*, 2019, pp. 223–230.
- [23] A. Mott, D. Zoran, M. Chrzanowski, D. Wierstra, and D. J. Rezende, "Towards interpretable reinforcement learning using attention augmented agents," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 12 350–12 359.
- [24] R. Zhang *et al.*, "AGIL: Learning attention from human for visuomotor tasks," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 663–679.
- [25] R. Zhang *et al.*, "Atari-HEAD: Atari human eye-tracking and demonstration dataset," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 6811–6820.
- [26] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, "The arcade learning environment: An evaluation platform for general agents," *J. Artif. Intell. Res.*, vol. 47, pp. 253–279, 2013.
- [27] M. Chevalier-Boisvert, F. Golemo, Y. Cao, B. Mehta, and L. Paull, "Duckietown environments for OpenAI Gym," 2018. [Online]. Available: <https://github.com/duckietown/gym-duckietown>
- [28] T. Dodson, N. Mattei, and J. Goldsmith, "A natural language argumentation interface for explanation generation in Markov decision processes," in *Proc. Int. Conf. Algorithmic Decis. Theory*, 2011, pp. 42–55.
- [29] F. Elizalde, L. E. Sucar, M. Luque, J. Diez, and A. Reyes, "Policy explanation in factored Markov decision processes," in *Proc. 4th Eur. Workshop Probabilistic Graphical Models*, 2008, pp. 97–104.
- [30] B. Hayes and J. A. Shah, "Improving robot controller transparency through autonomous policy explanation," in *Proc. 12th ACM/IEEE Int. Conf. Hum.-Robot Interact.*, 2017, pp. 303–312.
- [31] U. D. Gupta, E. Talvitie, and M. Bowling, "Policy tree: Adaptive representation for policy gradient," in *Proc. 29th AAAI Conf. Artif. Intell.*, 2015, pp. 2547–2553.
- [32] O. Bastani, Y. Pu, and A. Solar-Lezama, "Verifiable reinforcement learning via policy extraction," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 2494–2504.
- [33] A. M. Roth, N. Topin, P. Jamshidi, and M. Veloso, "Conservative Q-improvement: Reinforcement learning for an interpretable decision-tree policy," 2019, *arXiv:1907.01180*.
- [34] J. Waa, J. V. Diggelen, K. Bosch, and M. Neerinx, "Contrastive explanations for reinforcement learning in terms of expected consequences," in *Proc. Workshop Explainable AI IJCAI*, 2018.
- [35] P. Madumal, T. Miller, L. Sonenberg, and F. Vetere, "Explainable reinforcement learning through a causal lens," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 3, 2020, pp. 2493–2500.
- [36] L. V. D. Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. Nov., pp. 2579–2605, 2008.
- [37] Y. Engel and S. Mannor, "Learning embedded maps of Markov processes," in *Proc. Int. Conf. Mach. Learn.*, 2001, pp. 138–145.
- [38] K. Simonyan, A. Vedaldi, and A. Zisserman, "Deep inside convolutional networks: Visualising image classification models and saliency maps," in *Proc. Int. Conf. Learn. Representations*, 2014.
- [39] J. Zhang, S. A. Bargal, Z. Lin, J. Brandt, X. Shen, and S. Sclaroff, "Top-down neural attention by excitation backprop," *Int. J. Comput. Vis.*, vol. 126, no. 10, pp. 1084–1102, 2018.
- [40] D. Smilkov, N. Thorat, B. Kim, F. Viégas, and M. Wattenberg, "SmoothGrad: Removing noise by adding noise," 2017, *arXiv:1706.03825*.
- [41] L. M. Zintgraf, T. S. Cohen, T. Adel, and M. Welling, "Visualizing deep neural network decisions: Prediction difference analysis," in *Proc. Int. Conf. Learn. Representations*, 2017.
- [42] R. C. Fong and A. Vedaldi, "Interpretable explanations of black boxes by meaningful perturbation," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 3449–3457.
- [43] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 818–833.
- [44] V. Petsiuk, A. Das, and K. Saenko, "RISE: Randomized input sampling for explanation of black-box models," in *Proc. Brit. Mach. Vis. Conf.*, 2018, Art. no. 151.
- [45] L. S. Shapley, "A value for n-person games," *Contributions Theory Games*, vol. 2, no. 28, pp. 307–317, 1953.
- [46] M. Ancona, C. Öztireli, and M. Gross, "Explaining deep neural networks with a polynomial time algorithm for shapley values approximation," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 272–281.
- [47] R. Fong, M. Patrick, and A. Vedaldi, "Understanding deep networks via extremal perturbations and smooth masks," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 2950–2958.
- [48] P. Dabkowski and Y. Gal, "Real time image saliency for black box classifiers," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6967–6976.
- [49] W. Wang, J. Shen, X. Lu, S. C. H. Hoi, and H. Ling, "Paying attention to video object pattern understanding," *IEEE Trans. Pattern Anal. Mach. Intell.*, early access, Jan. 13, 2020, doi: [10.1109/TPAMI.2020.2966453](https://doi.org/10.1109/TPAMI.2020.2966453).
- [50] I. Sorokin, A. Seleznev, M. Pavlov, A. Fedorov, and A. Ignateva, "Deep attention recurrent Q-network," 2015, *arXiv:1512.01693*.
- [51] S. Mousavi, M. Schukat, E. Howley, A. Borji, and N. Mozayani, "Learning to predict where to look in interactive environments using deep recurrent Q-learning," 2016, *arXiv:1612.05753*.
- [52] Z. Yang, S. Bai, L. Zhang, and P. H. Torr, "Learn to interpret atari agents," 2018, *arXiv:1812.11276*.
- [53] D. Nikulin, A. Ianina, V. Aliev, and S. Nikolenko, "Free-lunch saliency via attention in atari agents," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshop*, 2019, pp. 4240–4249.
- [54] J. Choi, B.-J. Lee, and B.-T. Zhang, "Multi-focus attention network for efficient deep reinforcement learning," in *Proc. Workshops 31st AAAI Conf. Artif. Intell.*, 2017.
- [55] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [56] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
- [57] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1582–1591.
- [58] W. Shi, S. Song, and C. Wu, "Soft policy gradient method for maximum entropy deep reinforcement learning," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 3425–3431.
- [59] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. 13th AAAI Conf. Artif. Intell.*, 2016, pp. 2094–2100.
- [60] T. P. Lillicrap *et al.*, "Continuous control with deep reinforcement learning," in *Proc. Int. Conf. Learn. Representations*, 2016.
- [61] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. Int. Conf. Med. Image Comput. Comput.-Assisted Intervention*, 2015, pp. 234–241.
- [62] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017.
- [63] N. Mayer *et al.*, "A large dataset to train convolutional networks for disparity, optical flow, and scene flow estimation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 4040–4048.
- [64] G. Brockman *et al.*, "OpenAI Gym," 2016.
- [65] M. F. Land, "Vision, eye movements, and natural behavior," *Vis. Neurosci.*, vol. 26, no. 1, pp. 51–62, 2009.
- [66] G. Lin, A. Milan, C. Shen, and I. Reid, "RefineNet: Multi-path refinement networks for high-resolution semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 5168–5177.
- [67] S. Jégou, M. Drozdal, D. Vazquez, A. Romero, and Y. Bengio, "The one hundred layers Tiramisu: Fully convolutional DenseNets for semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, 2017, pp. 1175–1183.



Wenjie Shi received the BS degree from the School of Hydropower and Information Engineering, Huazhong University of Science and Technology, Wuhan, China, in 2016. He is currently working toward the PhD degree in control science and engineering from the Department of Automation, Institute of Industrial Intelligence and System, Tsinghua University, Beijing, China. His current research interests include deep reinforcement learning, interpretability, and underwater robot control.



Zhuoyuan Wang is currently working toward the BS degree in control science and engineering in the Department of Automation, Tsinghua University, Beijing, China. His current research interests include reinforcement learning, interpretability, and underwater robot control.



Gao Huang (Member, IEEE) received the BS degree from the School of Automation Science and Electrical Engineering, Beihang University, Beijing, China, in 2009, and the PhD degree from the Department of Automation, Tsinghua University, Beijing, China, in 2015. He was a visiting research scholar with the Department of Computer Science and Engineering, Washington University in St. Louis, St. Louis, Missouri, in 2013 and was a postdoctoral researcher with the Department of Computer Science, Cornell University, Ithaca, New York from 2015 to 2018. He is currently an assistant professor at the Department of Automation, Tsinghua University. His research interests include machine learning and deep learning.



Tingyu Lin received the BS and PhD degrees from the School of Automation Science and Electrical Engineering, Beihang University, Beijing, China, in 2007 and 2014, respectively. Since 2016, he has been a senior engineer with the State Key Laboratory of Intelligent Manufacturing System Technology, China. His current research interests include complex system modeling and simulation.



Shiji Song (Senior Member, IEEE) received the PhD degree in mathematics from the Department of Mathematics, Harbin Institute of Technology, Harbin, China, in 1996. He is currently a professor at the Department of Automation, Tsinghua University, Beijing, China. He has authored more than 180 research papers. His current research interests include system modeling, optimization and control, computational intelligence, and pattern recognition.



Cheng Wu received the BS and MS degrees in electrical engineering from Tsinghua University, Beijing, China. Since 1967, he has been with Tsinghua University, where he is currently a professor at the Department of Automation. He has also been an academican of the Chinese Academy of Engineering since 1995. His current research interests include manufacturing systems scheduling, optimization of supply chains, and system identification.

▷ For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.